

Section 6

Week 7 — Indexing II: Real Optimization Techniques

Schema: DVD Rental

Continues from: Week 6 — B-Tree, EXPLAIN ANALYZE, single vs composite indexes

Agenda

1. Selectivity — when is an index actually useful?
 2. Partial Index — index only the rows you care about
 3. Expression Index — index on a computed value
 4. Covering Index — query never touches the table
 5. Common Mistakes — what goes wrong in real systems
-

From Last Week

We covered the basics: what a B-Tree index is, the read/write trade-off, how to read EXPLAIN ANALYZE, and single vs composite indexes.

This week is about making **smarter decisions**. Creating an index is easy — creating the *right* index is the actual skill.

1. Selectivity

Selectivity = how unique are the values in a column?

- **High selectivity** → many distinct values → index is very useful
 - `customer_id`, `email`, `rental_id`
- **Low selectivity** → few distinct values → index is often ignored
 - `staff_id` (only 1 or 2), `activebool` (true/false)

When a column has very few distinct values, a query might still match thousands of rows. At that point, jumping through the index to fetch each one is actually slower than reading the table straight through. PostgreSQL knows this and will skip your index.

How to check before creating an index

```
SELECT COUNT(DISTINCT staff_id) AS distinct_values,
       COUNT(*)                AS total_rows
FROM rental;
```

```
distinct_values | total_rows
-----+-----
                2 |      16044
```

Only 2 values across 16,000 rows → low selectivity → index on `staff_id` is useless for filtering.

```
SELECT COUNT(DISTINCT customer_id) AS distinct_values,
       COUNT(*)                    AS total_rows
FROM rental;
```

```
distinct_values | total_rows
-----+-----
                599 |      16044
```

599 values → high selectivity → index will be used.

Demo

```
-- Create index on a low-selectivity column

CREATE INDEX idx_rental_staff ON rental (staff_id);

-- Query using that column

EXPLAIN ANALYZE

SELECT * FROM rental WHERE staff_id = 1;

-- PostgreSQL does a Seq Scan – index is ignored

-- Compare with a high-selectivity column

EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;

-- Index Scan – PostgreSQL uses the existing index immediately
```

```
-- Cleanup
```

```
DROP INDEX IF EXISTS idx_rental_staff;
```

Practice — Part A

Before creating any index, always check if the column is worth indexing. Use the `payment` table to measure selectivity of `staff_id`, create an index on it, then run the query and observe whether PostgreSQL actually uses it.

```
-- 1. Check selectivity of staff_id in the payment table
```

```
SELECT COUNT(DISTINCT staff_id) AS distinct_values,  
  
       COUNT(*)                AS total_rows  
  
FROM payment;
```

```
-- 2. Create an index on payment.staff_id
```

```
CREATE INDEX idx_payment_staff ON payment (staff_id);
```

```
-- 3. Run the query and look at the plan
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM payment WHERE staff_id = 1;
```

```
-- Cleanup
```

```
DROP INDEX IF EXISTS idx_payment_staff;
```

Answers:

- `staff_id` has 2 distinct values across ~14,000 rows → very low selectivity.
- PostgreSQL does a **Seq Scan** — it sees that half the table matches and decides jumping through the index is not worth it.
- The index exists but is ignored. This is expected and correct behavior.

Practice — Part B

A very common real-world mistake is indexing a boolean column expecting to speed up "show all active records" queries. Check the distribution of values in `customer.activebool`, create a regular index, then run a query for `true` and see what PostgreSQL decides.

```
-- 1. How many active vs inactive customers?
```

```
SELECT activebool, COUNT(*) AS total

FROM customer

GROUP BY activebool;


-- 2. Create an index on the boolean column

CREATE INDEX idx_customer_active ON customer (activebool);


-- 3. Query for all active customers

EXPLAIN ANALYZE

SELECT * FROM customer WHERE activebool = true;


-- Cleanup

DROP INDEX IF EXISTS idx_customer_active;
```

Answers:

- Almost all customers have `activebool = true` — maybe 1 or 2 are inactive.
- PostgreSQL does a **Seq Scan** — the index would point to 99% of all rows, which means visiting almost every page anyway. Reading them in order is cheaper.
- An index on a boolean column is only useful for the **rare value** — and even then, a partial index (Topic 2) is the right tool.

2. Partial Index

A **partial index** only includes rows that match a WHERE condition. It's smaller and faster than a full index because it only covers the subset of data you actually query.

```
CREATE INDEX index_name ON table (column)

WHERE condition;
```

When to use it: Your queries always include the same filter — a status check, a NULL check, a flag. Instead of indexing 16,000 rows, index only the 2,000 that matter.

Demo

```
-- See how many rentals are still active (not returned yet)

SELECT COUNT(*) FROM rental WHERE return_date IS NULL;
```

```

-- Query before index

EXPLAIN ANALYZE

SELECT * FROM rental

WHERE customer_id = 148

    AND return_date IS NULL;


-- Create a partial index – only for unreturned rentals

CREATE INDEX idx_rental_active_customer

    ON rental (customer_id)

WHERE return_date IS NULL;


-- Same query after index

EXPLAIN ANALYZE

SELECT * FROM rental

WHERE customer_id = 148

    AND return_date IS NULL;


-- Cleanup

DROP INDEX IF EXISTS idx_rental_active_customer;

```

The partial index is much smaller than a full index on `customer_id`, but answers this exact query pattern perfectly.

Connecting to Selectivity — Boolean Columns

Remember from Topic 1 that an index on `activebool` was useless for `activebool = true` because 99% of rows match. This is where a partial index becomes the right tool.

If you actually need to find inactive customers (the rare case), index only those:

```

-- How many inactive customers exist?

SELECT COUNT(*) FROM customer WHERE activebool = false;


-- Query for inactive customers – no index yet

```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE activebool = false;
```

```
-- Seq Scan - small table but no index
```

```
-- Create a partial index for the rare value only
```

```
CREATE INDEX idx_customer_inactive
```

```
    ON customer (customer_id)
```

```
    WHERE activebool = false;
```

```
-- Same query with index
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE activebool = false;
```

```
-- Index Scan  - only covers the few inactive rows
```

```
-- Cleanup
```

```
DROP INDEX IF EXISTS idx_customer_inactive;
```

The full index covered 599 customers. This partial index covers maybe 2 or 3 — it's tiny, and for a query that returns only those few rows, it's perfect.

A partial index on a boolean column makes sense only for the **rare value**. Never for the common one.

Practice

Scenario: staff member 2 runs a daily report that always filters for unreturned rentals. Run the query first to see the baseline plan, then design and create a partial index that fits this exact access pattern. Re-run the query and compare the execution time and scan type.

```
-- 1. Run the query and note the plan
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM rental
```

```
WHERE staff_id = 2
```

```
    AND return_date IS NULL;
```

```
-- 2. Create a partial index for this query pattern
```

```
--      (Which column to index? What condition to use?)

-- 3. Run the same query again and compare

-- 4. Drop your index
```

Answers:

- **The index:** `CREATE INDEX ON rental (staff_id) WHERE return_date IS NULL;`
- Before: Seq Scan — no usable index, plus `staff_id` has low selectivity on the full table.
- After: Index Scan using the partial index — it only covers the unreturned rows (~3,000), making `staff_id = 2` much more selective within that smaller subset.
- **Why better than a full index on `staff_id`:** A full index covers all 16,000 rows and gets ignored due to low selectivity. The partial index covers ~3,000 rows, so the match ratio is much better.
- **When the partial index would NOT be used:** Any query that doesn't include `return_date IS NULL` . The condition must match exactly.

3. Expression Index

When you apply a **function** to a column in the WHERE clause, a regular index on that column is useless. The index stores original values; the query searches transformed values — they don't match.

```
-- Regular index on email cannot help here:

WHERE lower(email) = 'john@example.com'
```

PostgreSQL has to compute `lower(email)` on every row. It can't use the B-Tree because the index doesn't store lowercase values.

Fix: Create an index that stores the pre-computed result.

```
CREATE INDEX index_name ON table (expression);
```

Common cases:

Query pattern	Index expression
Case-insensitive email search	<code>lower(email)</code>
Filter by year only	<code>EXTRACT(YEAR FROM rental_date)</code>
Uppercase normalize	<code>upper(last_name)</code>

The query must use the exact same expression — otherwise PostgreSQL won't recognize it.

Demo

```
-- Search without expression index

EXPLAIN ANALYZE

SELECT * FROM customer

WHERE lower(email) = 'mary.smith@sakilacustomer.org';

-- Seq Scan – PostgreSQL has to compute lower() for every row


-- Create the expression index

CREATE INDEX idx_customer_email_lower ON customer (lower(email));


-- Same query now

EXPLAIN ANALYZE

SELECT * FROM customer

WHERE lower(email) = 'mary.smith@sakilacustomer.org';

-- Index Scan – the lowercase value is already stored in the index


-- Cleanup

DROP INDEX IF EXISTS idx_customer_email_lower;
```

Practice

Scenario: A report always queries rentals by year only — not the full date. Run the query first and note that a regular index on `rental_date` won't help here. Then create the right expression index so the query uses it. The expression in your index must exactly match the expression in the WHERE clause.

```
-- 1. Run the query and check the plan

EXPLAIN ANALYZE

SELECT * FROM rental

WHERE EXTRACT(YEAR FROM rental_date) = 2005;


-- 2. Create an expression index (the expression must match the WHERE clause exact
```



```
-- 3. Re-run and compare the plan
```

```
-- 4. Drop your index
```

Answers:

- **The index:** `CREATE INDEX ON rental (EXTRACT(YEAR FROM rental_date));`
- Before: Seq Scan — PostgreSQL computes `EXTRACT(YEAR ...)` on all 16,000 rows one by one.
- After: Index Scan — the pre-computed year is stored in the index, so the lookup is instant.
- **Why doesn't a regular index on `rental_date` help?** The index stores full timestamps like `2005-05-25 11:30:37`. The query asks for `EXTRACT(YEAR FROM rental_date) = 2005` which is a different value — PostgreSQL can't map between them.
- **If the query used `DATE_TRUNC` instead:** The index would need to match exactly — `CREATE INDEX ON rental (DATE_TRUNC('year', rental_date))`. A `EXTRACT`-based index would not be used.

4. Covering Index

When PostgreSQL uses a regular index, it does two things:

1. Reads the index to find which rows match
2. Goes to the actual table to fetch the full row

A **covering index** stores extra columns inside the index itself. If all the columns the query needs are in the index, PostgreSQL never touches the table at all — this is called an **Index Only Scan**.

```
CREATE INDEX index_name ON table (column_to_search)

INCLUDE (extra_column_1, extra_column_2);
```

- Columns in `(...)` → used for searching and sorting
- Columns in `INCLUDE (...)` → stored alongside, not sorted

Demo

```
-- Query that needs 3 columns from rental
```

```
EXPLAIN ANALYZE
```

```
SELECT customer_id, rental_date, return_date
```

```
FROM rental
```

```
WHERE customer_id = 148;
```

```
-- Bitmap Heap Scan - index used to find rows, then table accessed for the data
```

```
-- Create a covering index with those columns included

CREATE INDEX idx_rental_covering

    ON rental (customer_id)

INCLUDE (rental_date, return_date);


-- IMPORTANT: run VACUUM before testing – explained below

VACUUM rental;


-- Same query

EXPLAIN ANALYZE

SELECT customer_id, rental_date, return_date

FROM rental

WHERE customer_id = 148;


-- Index Only Scan  – no table access at all


-- Add one column that's not in the index

EXPLAIN ANALYZE

SELECT customer_id, rental_date, return_date, inventory_id

FROM rental

WHERE customer_id = 148;


-- Falls back to Bitmap Heap Scan – inventory_id isn't in the index


-- Cleanup

DROP INDEX IF EXISTS idx_rental_covering;
```

Practice

Scenario: A dashboard always runs this exact query on the payment table — it only needs 3 columns. Your goal is to design a covering index so PostgreSQL can answer this query entirely from the index without touching the table. After creating it, verify you get an Index Only Scan. Then add a 4th column to the SELECT and observe what happens to the plan.

```
-- 1. Run and check the plan
```

```
EXPLAIN ANALYZE
```

```
SELECT customer_id, amount, payment_date
```

```
FROM payment
```

```
WHERE customer_id = 341;
```

```
-- 2. Create a covering index for this exact query
```

```
-- 3. Verify you get an Index Only Scan
```

```
-- 4. Now add staff_id to the SELECT – does the plan change?
```

```
EXPLAIN ANALYZE
```

```
SELECT customer_id, amount, payment_date, staff_id
```

```
FROM payment
```

```
WHERE customer_id = 341;
```

```
-- 5. Drop your index
```

Answers:

- **The index:** `CREATE INDEX ON payment (customer_id) INCLUDE (amount, payment_date);`
- Step 3: **Index Only Scan** — PostgreSQL gets `customer_id`, `amount`, and `payment_date` directly from the index without touching the table.
- Step 4: Falls back to **Bitmap Heap Scan** — `staff_id` is not stored in the index, so PostgreSQL must go to the table to fetch it.
- **Bitmap Heap Scan vs Index Only Scan:** Bitmap Heap Scan uses the index to find row locations, then fetches the actual data from the table (two operations). Index Only Scan reads everything from the index alone (one operation — faster).
- **When not worth it:** When the query needs many columns — including all of them in the index wastes a lot of disk space. Best for narrow, frequently-run queries that return only a few columns.

5. Common Mistakes

These are the mistakes you'll see most often in real codebases.

Mistake 1 — Indexing a low-selectivity column

```
CREATE INDEX ON rental (staff_id); -- only 2 values, useless
```

Fix: Don't index it alone. Combine with a high-selectivity column if needed, or use a partial index.

Mistake 2 — Wrong column order in composite index

```
CREATE INDEX ON rental (rental_date, customer_id);

-- This will NOT use the index - customer_id is not the leading column

EXPLAIN ANALYZE

SELECT * FROM rental WHERE customer_id = 148;

-- Seq Scan ❌
```

Fix: Put the column you filter on most (or most selectively) first.

Mistake 3 — Function on a column breaks the index

```
-- Index exists on last_name - but this query ignores it

SELECT * FROM customer WHERE upper(last_name) = 'SMITH';

-- Seq Scan ` ← ignored
```

Fix: Create an expression index — `CREATE INDEX ON customer (upper(last_name));`

Mistake 4 — Too many indexes on a write-heavy table

Every `INSERT`, `UPDATE`, and `DELETE` must update all indexes on that table. 10 indexes = 10× more write overhead.

Fix: Only keep indexes you actually use. Check with:

```
SELECT indexrelname, idx_scan

FROM pg_stat_user_indexes

WHERE schemaname = 'public'

AND idx_scan = 0;

-- idx_scan = 0 means this index has never been used since last stats reset
```

Mistake 5 — Forgetting ANALYZE after bulk loads

This is the hardest mistake to spot — everything looks fine, the index exists, the query is correct, but PostgreSQL still picks the wrong plan.

The cause: stale statistics.

Before running any query, PostgreSQL's planner estimates how many rows will match. It doesn't look at the actual data — it reads from a statistics table called `pg_statistic` that holds a pre-computed summary of your table.

What goes wrong when stats are stale:

```
Stats say:    rental has 16,000 rows, customer_id = 148 → ~30 rows

Reality:      rental now has 500,000 rows, customer_id = 148 → ~1,500 rows
```

The planner picks an Index Scan expecting 30 rows. It gets 1,500 — and jumping to 1,500 rows via index is slower than a straight Seq Scan. Wrong plan, worse performance.

How to spot it in EXPLAIN ANALYZE:

```
Seq Scan on rental  (cost=0.00..310.44 rows=30 width=36)

      (actual time=0.021..45.2 ms rows=1500 loops=1)
```

`rows=30` (estimate) vs `rows=1500` (actual) — 50× off. When estimate and reality differ by more than 5–10×, stats are stale.

Why PostgreSQL doesn't fix this automatically:

PostgreSQL has a background process (autovacuum) that runs ANALYZE automatically — but only after 20% of the table has changed. On a 1,000,000 row table, that's 200,000 rows of change before stats update. During bulk loads, stats can be very wrong for a long time.

Fix:

```
-- After any bulk INSERT or TRUNCATE + reload:

ANALYZE rental;

-- For the whole database:

ANALYZE;
```

Demo

```
-- Step 1: Check existing indexes on customer
```

```
SELECT indexname, indexdef FROM pg_indexes
```

```
WHERE tablename = 'customer';
```

```
-- Step 2: Query with upper() – does the existing index help?
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE upper(last_name) = 'SMITH';
```

```
-- Seq Scan ❌
```

```
-- Step 3: Create the right expression index
```

```
CREATE INDEX idx_customer_upper_last ON customer (upper(last_name));
```

```
-- Step 4: Same query
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE upper(last_name) = 'SMITH';
```

```
-- Index Scan ✅
```

```
-- Cleanup
```

```
DROP INDEX IF EXISTS idx_customer_upper_last;
```

Practice

Each query below has a different indexing problem. For each one: run `EXPLAIN ANALYZE` first and note the scan type, then figure out what's causing it and what type of index would fix it. Create the index, re-run the query, and confirm the plan improved. After each task, drop the index before moving to the next.

```
-- Query A
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE activebool = true;
```

```
-- Query B
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM customer WHERE lower(email) LIKE 'a%';
```

```
-- Query C - create this first, then run the query

CREATE INDEX idx_test_composite ON rental (customer_id, rental_date);

EXPLAIN ANALYZE

SELECT * FROM rental WHERE rental_date > '2005-08-01';

-- Cleanup

DROP INDEX IF EXISTS idx_test_composite;
```

Answers:

Query A — activebool = true

- Scan type: **Seq Scan**
- Why: activebool has only 2 values and almost all rows are true — low selectivity. An index would point to 99% of the table, which is worse than reading straight through.
- Fix: No useful regular index here. Use a partial index for the rare case: CREATE INDEX ON customer (customer_id) WHERE activebool = false;

Query B — lower(email) LIKE 'a%'

- Scan type: **Seq Scan**
- Why: There's no index on lower(email) . Even if there were an index on email , applying lower() breaks it — the stored values and the searched values are different.
- Fix: CREATE INDEX ON customer (lower(email)); — then the query can use it for prefix matches like LIKE 'a%' .

Query C — rental_date > '2005-08-01'

- Scan type: **Seq Scan** (index ignored)
- Why: The composite index is (customer_id, rental_date) . rental_date is not the leading column — the index is sorted by customer_id first. Without a customer_id filter, PostgreSQL can't use it.
- Fix: Create a separate index on rental_date alone: CREATE INDEX ON rental (rental_date);

Key Takeaways

Topic	What to remember
Selectivity	Check distinct values before creating an index. Low selectivity = ignored.
Partial Index	Always filter the same condition? Put it in the index with WHERE.
Expression Index	Function in WHERE? The index must store the same expression.
Covering Index	INCLUDE the columns SELECT needs → Index Only Scan → no table access.
Common Mistakes	Wrong column order, low selectivity, functions breaking indexes, stale stats.

The diagnostic loop — use it every time:

Slow query → EXPLAIN ANALYZE → find the problem → pick the right index → verify

